

B R Codes

B.1 Local asymptotic power

```
#####  
## Local asymptotic power of the proposed CVM test  
#####  
set.seed(123)  
alpha <- 0.05  
B <- 100000          # Monte Carlo replications  
Delta <- 0.80         # efficacy constant (>0)  
theta <- seq(0,4,by=0.5)  
## Null distribution  
T0 <- rnorm(B)^2  
## Critical value  
c.alpha <- quantile(T0,1-alpha)  
## Local power  
power <- numeric(length(theta))  
for(i in seq_along(theta)){  
  Z <- rnorm(B)  
  T.theta <- (Z + theta[i]*sqrt(Delta))^2  
  power[i] <- mean(T.theta > c.alpha)  
}  
results <- data.frame(theta,power)  
print(results)  
plot(theta,  
      power,  
      type="b",  
      pch=19,  
      lwd=2,  
      ylim=c(0,1),  
      xlab=expression(theta),  
      ylab="Estimated Local Power")  
abline(h=0.05,col=2,lty=2)  
#####  
## Pitman Asymptotic Relative Efficiency  
#####  
Delta.CVM <- 0.80  
Delta0 <- 0.60  
ARE <- Delta.CVM/Delta0  
ARE  
Delta.CVM <- c(0.60,0.65,0.70,0.75,0.80,  
              0.84,0.87,0.89,0.90)
```

```

Delta0 <- rep(0.60,length(Delta.CVM))
ARE <- Delta.CVM/Delta0
data.frame(theta,ARE)

#####
## Monte Carlo Power Study versus Polynomial Degree r
## Empirical CvM Statistic based on Polynomial Approximation
#####

set.seed(12345)

#####
## Simulation Parameters
#####

n      <- 300          # Sample size
R      <- 500          # Monte Carlo replications
B      <- 300          # Bootstrap replications
alpha  <- 0.05         # Significance level
theta  <- 1.5          # Dependence parameter
d      <- 1            # Finite difference order

r.grid <- 1:10         # Polynomial degrees

#####
## Data Generation
#####

generate.data <- function(n, theta){

  Z <- runif(n,-1,1)

  eps <- rnorm(n)

  ## Alternative hypothesis
  eps <- eps + theta*sin(pi*Z)

  ## Response
  Y <- sin(2*pi*Z) + eps

  list(Y=Y,Z=Z)
}

```

```

}

#####
## Polynomial Approximation
#####

poly.map <- function(z,r){

  out <- rep(0,length(z))

  for(k in 0:r){

    out <- out + z^(2*k+1)/factorial(2*k+1)

  }

  return(out)

}

#####
## Proposed Empirical CvM Statistic
#####

compute.statistic <- function(Y,Z,r,d){

  #####
  ## Polynomial approximation
  #####

  U <- poly.map(Z,r)

  #####
  ## Finite difference
  #####

  W <- diff(Y,differences=d)

  U <- U[(d+1):length(U)]

  n1 <- length(U)

```

```
#####
## Marginal empirical CDFs
#####

FnU <- rank(U)/(n1+1)

FnW <- rank(W)/(n1+1)

#####
## Empirical Process
#####

G <- numeric(n1)

for(i in 1:n1){

  Fn <- mean(U<=U[i] & W<=W[i])

  G[i] <- sqrt(n1)*(Fn-FnU[i]*FnW[i])

}

#####
## Empirical CvM Statistic #####
T <- mean(G^2)
return(T)
}

#####
## Bootstrap Critical Value
#####
bootstrap.cv <- function(r,B=300){
  stat <- numeric(B)
  for(b in 1:B){
    dat <- generate.data(n,0)
    stat[b] <- compute.statistic(dat$Y,
                                dat$Z,
                                r,
                                d)

  }
  return(as.numeric(quantile(stat,1-alpha)))
}
#####
```

```

## Critical Values
#####
cat("Computing Bootstrap Critical Values...\n")
crit <- numeric(length(r.grid))
for(i in seq_along(r.grid)){
  crit[i] <- bootstrap.cv(r.grid[i],B)
}
#####
## Monte Carlo Power
#####
cat("Computing Empirical Power...\n")
power <- numeric(length(r.grid))
for(j in seq_along(r.grid)){
  r <- r.grid[j]

  reject <- numeric(R)

  for(rep in 1:R){

    dat <- generate.data(n,theta)

    Tstat <- compute.statistic(dat$Y,
                              dat$Z,
                              r,
                              d)

    reject[rep] <- (Tstat > crit[j])

  }
  power[j] <- mean(reject)
}
#####
## Results
#####
Power.Table <- data.frame(
  Degree = r.grid,
  Power = round(power,3)
)
print(Power.Table)
#####
## Power Plot
#####

```

```

plot(r.grid,
     power,
     type="b",
     pch=19,
     lwd=2,
     col="blue",
     xlab="Polynomial Degree (r)",
     ylab="Empirical Power",
     ylim=c(0,1),
     main="Power versus Polynomial Degree")
grid()
#####
## Save Results
#####
write.csv(Power.Table,
          "Power_vs_Polynomial_Degree.csv",
          row.names=FALSE)
#####
## Estimated Asymptotic Relative Efficiency
#####
R <- 1000
ARE <- numeric(length(r.grid))
#####
## Reference statistic
#####
reference.statistic <- function(Y,Z){
  ## Classical empirical CvM statistic
  n <- length(Y)
  Fy <- rank(Y)/(n+1)
  Fz <- rank(Z)/(n+1)
  G <- numeric(n)
  for(i in 1:n){
    Fn <- mean(Y<=Y[i] & Z<=Z[i])
    G[i] <- sqrt(n)*(Fn-Fy[i]*Fz[i])
  }
  mean(G^2)
}
#####
## Estimated ARE
#####
for(j in seq_along(r.grid)){
  r <- r.grid[j]

```

```

prop <- numeric(R)
ref <- numeric(R)
for(rep in 1:R){
  dat <- generate.data(n,theta)
  prop[rep] <-
    compute.statistic(dat$Y,
                      dat$Z,
                      r,
                      d)

  ref[rep] <-
    reference.statistic(dat$Y,
                       dat$Z)

}
var.prop <- var(prop)
var.ref <- var(ref)
ARE[j] <- var.ref/var.prop
}
#####
## Results
#####
ARE.Table <- data.frame(
  Degree=r.grid,
  ARE=round(ARE,3)
)
print(ARE.Table)

```

B.2 Simulation studies

```

#####
# Empirical Size of the Proposed Cramér--von Mises Test
# for Different Polynomial Degrees (r) and Difference
# Orders (d)
#####

## -----
## 1. Packages
## -----
library(MASS)

## -----
## 2. Simulation settings

```

```

## -----

set.seed(12345)

# Sample sizes
n_values <- c(100, 200, 300, 500)

# Polynomial degrees to be investigated
r_values <- 1:10

# Difference orders to be investigated
d_values <- 1:4

# Nominal significance level
alpha <- 0.05

# Number of Monte Carlo replications
R <- 1000

# Number of bootstrap replications
B <- 500

## -----
## 3. Data-generating mechanism under H0
## -----
# IMPORTANT:
# Replace the distributions/functional form below by the
# exact data-generating mechanism used in your manuscript.
#
# Under H0:
#     epsilon independent of Z
#
#      $Y = X' \beta + \phi(V) + \epsilon$ 
#
# The structure below is only the computational framework.

generate_data <- function(n) {

  ## Parametric covariate(s)
  X <- matrix(rnorm(n), ncol = 1)

```



```

## Nonparametric covariate(s)
V <- matrix(runif(n, -1, 1), ncol = 1)

## Parameter
beta <- 1

## Smooth component
phi <- sin(pi * V[, 1])

## Error under H0:
## independent of X and V
epsilon <- rnorm(n, mean = 0, sd = 1)

## Response
Y <- as.vector(X %*% beta + phi + epsilon)

list(
  Y = Y,
  X = X,
  V = V,
  Z = cbind(X, V),
  epsilon = epsilon
)
}

## -----
## 4. Complete polynomial basis
## -----
# For the present one-dimensional X and one-dimensional V,
# construct all monomials  $X^a V^b$  satisfying  $a+b \leq r$ .

polynomial_basis <- function(X, V, r) {

  X <- as.vector(X)
  V <- as.vector(V)

  basis <- list()
  names_basis <- character(0)

  k <- 1

```

```

for (a in 0:r) {
  for (b in 0:(r - a)) {

    basis[[k]] <- X^a * V^b
    names_basis[k] <- paste0("X", a, "V", b)

    k <- k + 1
  }
}

basis <- as.data.frame(basis)
names(basis) <- names_basis

as.matrix(basis)
}

## -----
## 5. Polynomial approximation P_r(Z)
## -----

get_polynomial_approximation <- function(X, V, Y, r) {

  P <- polynomial_basis(X, V, r)

  # Least-squares polynomial approximation
  fit <- lm.fit(x = P, y = Y)

  # Numerical protection against singular/ill-conditioned
  # polynomial designs
  beta_hat <- fit$coefficients
  beta_hat[is.na(beta_hat)] <- 0

  U <- as.vector(P %*% beta_hat)

  list(
    U = U,
    coefficients = beta_hat,
    basis = P
  )
}

```

```

## -----
## 6. d-th order forward difference
## -----

forward_difference <- function(Y, d) {

  if (d == 0)
    return(Y)

  n <- length(Y)

  if (n <= d)
    stop("Difference order d must be smaller than sample size.")

  W <- Y

  for (j in 1:d) {
    W <- diff(W)
  }

  as.vector(W)
}

## -----
## 7. Align U with differenced response W
## -----
# After differencing, W has length n-d.
# We retain the corresponding observations from U.

construct_UW <- function(U, Y, d) {

  W <- forward_difference(Y, d)

  # For a forward difference,
  #  $\Delta^d Y_i$  uses  $Y_i, \dots, Y_{i+d}$ .
  # Hence U is aligned with its initial n-d observations.

  U_aligned <- U[1:length(W)]

  list(

```

```

    U = U_aligned,
    W = W
  )
}

## -----
## 8. Cramér--von Mises independence statistic
## -----

cvm_statistic <- function(U, W) {

  n <- length(U)

  ## Empirical joint distribution evaluated at
  ## each observed pair.
  F_joint <- numeric(n)

  F_U <- numeric(n)
  F_W <- numeric(n)

  for (i in 1:n) {

    F_joint[i] <-
      mean(U <= U[i] & W <= W[i])

    F_U[i] <-
      mean(U <= U[i])

    F_W[i] <-
      mean(W <= W[i])
  }

  G <- sqrt(n) * (F_joint - F_U * F_W)

  ## Integral with respect to the empirical joint measure
  mean(G^2)
}

## -----
## 9. Bootstrap critical value under H0

```

```
## -----

bootstrap_critical_value <- function(U, W, alpha = 0.05, B = 500) {

  n <- length(U)

  T_boot <- numeric(B)

  for (b in 1:B) {

    ## Independently resample the two empirical marginals
    U_star <- sample(U, size = n, replace = TRUE)
    W_star <- sample(W, size = n, replace = TRUE)

    T_boot[b] <- cvm_statistic(U_star, W_star)
  }

  quantile(
    T_boot,
    probs = 1 - alpha,
    names = FALSE,
    type = 8
  )
}
```

```
## -----
## 10. One Monte Carlo replication
## -----
```

```
one_replication <- function(n, r, d, alpha = 0.05, B = 500) {

  ## Generate data under H0
  dat <- generate_data(n)

  ## Polynomial approximation
  poly <- get_polynomial_approximation(
    X = dat$X,
    V = dat$V,
    Y = dat$Y,
    r = r
  )
}
```

```

U <- poly$U

## Construct differenced response
UW <- construct_UW(
  U = U,
  Y = dat$Y,
  d = d
)

U <- UW$U
W <- UW$W

## Observed statistic
T_obs <- cvm_statistic(U, W)

## Bootstrap critical value
c_alpha <- bootstrap_critical_value(
  U = U,
  W = W,
  alpha = alpha,
  B = B
)

## Rejection indicator
reject <- as.integer(T_obs > c_alpha)

c(
  statistic = T_obs,
  critical_value = c_alpha,
  reject = reject
)
}

## -----
## 11. Empirical size for one (n, r, d) combination
## -----

estimate_size <- function(n, r, d,
  R = 1000,
  alpha = 0.05,

```

```

      B = 500) {

rejection <- numeric(R)

for (rep in 1:R) {

  result <- one_replication(
    n = n,
    r = r,
    d = d,
    alpha = alpha,
    B = B
  )

  rejection[rep] <- result["reject"]

}

empirical_size <- mean(rejection)

## Monte Carlo standard error
MCSE <- sqrt(
  empirical_size * (1 - empirical_size) / R
)

## Monte Carlo confidence interval
lower <- empirical_size -
  qnorm(0.975) * MCSE

upper <- empirical_size +
  qnorm(0.975) * MCSE

data.frame(
  n = n,
  r = r,
  d = d,
  alpha = alpha,
  R = R,
  empirical_size = empirical_size,
  MCSE = MCSE,
  lower_95 = max(0, lower),
  upper_95 = min(1, upper)

```

```

    )
}

## -----
## 12. Run the complete (n, r, d) experiment
## -----

size_results <- list()

counter <- 1

for (n in n_values) {

  for (r in r_values) {

    for (d in d_values) {

      cat(
        "Running:",
        "n =", n,
        "r =", r,
        "d =", d, "\n"
      )

      size_results[[counter]] <-
        estimate_size(
          n = n,
          r = r,
          d = d,
          R = R,
          alpha = alpha,
          B = B
        )

      counter <- counter + 1
    }
  }
}

## -----

```



```

## 13. Combine all results
## -----

size_results <- do.call(
  rbind,
  size_results
)

rownames(size_results) <- NULL

## -----
## 14. Display empirical sizes
## -----

print(size_results)

## -----
## 15. Save results
## -----

write.csv(
  size_results,
  file = "empirical_size_r_d.csv",
  row.names = FALSE
)

## -----
## 16. Empirical-size tables for each difference order
## -----

for (d in d_values) {

  cat("\n=====\n")
  cat("Difference order d =", d, "\n")
  cat("=====\n")

  temp <- subset(
    size_results,
    d == d
  )
}

```

```

)

print(temp)
}

## -----
## 17. Size matrix for a selected sample size
## -----

n_selected <- n_values[1]

size_matrix <- reshape(
  subset(size_results, n == n_selected),
  idvar = "r",
  timevar = "d",
  direction = "wide"
)

print(size_matrix)

## -----
## 18. Optional: size distortion
## -----

size_results$size_distortion <-
  size_results$empirical_size - alpha

size_results$absolute_size_distortion <-
  abs(size_results$size_distortion)

write.csv(
  size_results,
  file = "empirical_size_with_distortion.csv",
  row.names = FALSE
)

```

B.3 Asymptotic powers (to run later)

```

#####
# Empirical Power of the Proposed Cramer--von Mises Test
# for Different Polynomial Degrees (r) and Difference

```

```

# Orders (d)
#####

library(MASS)

## -----
## 1. Simulation settings
## -----

set.seed(12345)

n_values <- c(100, 200, 300, 500)

r_values <- 1:10

d_values <- 1:4

alpha <- 0.05

# Number of Monte Carlo replications
R <- 1000

# Number of bootstrap replications
B <- 500

# Local-alternative parameter
theta_values <- c(0.5, 1, 1.5, 2)

## -----
## 2. Data-generating mechanism under H1
## -----

generate_data <- function(n, theta) {

  ## Parametric covariate
  X <- matrix(rnorm(n), ncol = 1)

  ## Nonparametric covariate
  V <- matrix(runif(n, -1, 1), ncol = 1)

  ## Parameter

```

```

beta <- 1

## Smooth component
phi <- sin(pi * V[, 1])

## Independent error component
eta <- rnorm(n, mean = 0, sd = 1)

## Local departure from independence
t <- theta / sqrt(n)

## Covariate-dependent component
g <- sin(pi * V[, 1])

## Error under H1
epsilon <- eta + t * g

## Response
Y <- as.vector(
  X %*% beta + phi + epsilon
)

list(
  Y = Y,
  X = X,
  V = V,
  Z = cbind(X, V),
  epsilon = epsilon
)
}

## -----
## 3. Complete polynomial basis
## -----

polynomial_basis <- function(X, V, r) {

  X <- as.vector(X)
  V <- as.vector(V)

  basis <- list()

```

```

names_basis <- character(0)

k <- 1

for (a in 0:r) {

  for (b in 0:(r - a)) {

    basis[[k]] <- X^a * V^b

    names_basis[k] <-
      paste0("X", a, "V", b)

    k <- k + 1
  }
}

basis <- as.data.frame(basis)

names(basis) <- names_basis

as.matrix(basis)
}

## -----
## 4. Polynomial approximation P_r(Z)
## -----

get_polynomial_approximation <- function(X, V, Y, r) {

  P <- polynomial_basis(X, V, r)

  fit <- lm.fit(
    x = P,
    y = Y
  )

  beta_hat <- fit$coefficients

  beta_hat[is.na(beta_hat)] <- 0

```

```

U <- as.vector(
  P %*% beta_hat
)

list(
  U = U,
  coefficients = beta_hat,
  basis = P
)
}

## -----
## 5. d-th order forward difference
## -----

forward_difference <- function(Y, d) {

  if (d == 0)
    return(Y)

  n <- length(Y)

  if (n <= d)
    stop(
      "Difference order d must be smaller than sample size."
    )

  W <- Y

  for (j in 1:d) {

    W <- diff(W)
  }

  as.vector(W)
}

## -----
## 6. Align U with differenced response W
## -----

```

```

construct_UW <- function(U, Y, d) {

  W <- forward_difference(Y, d)

  U_aligned <- U[1:length(W)]

  list(
    U = U_aligned,
    W = W
  )
}

## -----
## 7. Cramer--von Mises independence statistic
## -----

cvm_statistic <- function(U, W) {

  n <- length(U)

  F_joint <- numeric(n)

  F_U <- numeric(n)

  F_W <- numeric(n)

  for (i in 1:n) {

    F_joint[i] <-
      mean(
        U <= U[i] &
        W <= W[i]
      )

    F_U[i] <-
      mean(
        U <= U[i]
      )

    F_W[i] <-

```

```

    mean(
      W <= W[i]
    )
  }

G <-
  sqrt(n) *
  (
    F_joint -
      F_U * F_W
  )

mean(G^2)
}

## -----
## 8. Bootstrap critical value
## -----

bootstrap_critical_value <- function(
  U,
  W,
  alpha = 0.05,
  B = 500
) {

  n <- length(U)

  T_boot <- numeric(B)

  for (b in 1:B) {

    ## Independent resampling from
    ## empirical marginal distributions
    U_star <-
      sample(
        U,
        size = n,
        replace = TRUE
      )
  }

```



```

W_star <-
  sample(
    W,
    size = n,
    replace = TRUE
  )

T_boot[b] <-
  cvm_statistic(
    U_star,
    W_star
  )
}

quantile(
  T_boot,
  probs = 1 - alpha,
  names = FALSE,
  type = 8
)
}

## -----
## 9. One Monte Carlo replication under H1
## -----

one_replication <- function(
  n,
  r,
  d,
  theta,
  alpha = 0.05,
  B = 500
) {

  ## Generate data under H1
  dat <-
    generate_data(
      n = n,
      theta = theta
    )

```

```

## Polynomial approximation
poly <-
  get_polynomial_approximation(
    X = dat$X,
    V = dat$V,
    Y = dat$Y,
    r = r
  )

U <- poly$U

## Construct differenced response
UW <-
  construct_UW(
    U = U,
    Y = dat$Y,
    d = d
  )

U <- UW$U
W <- UW$W

## Observed statistic
T_obs <-
  cvm_statistic(
    U,
    W
  )

## Bootstrap critical value
c_alpha <-
  bootstrap_critical_value(
    U = U,
    W = W,
    alpha = alpha,
    B = B
  )

## Rejection indicator
reject <-
  as.integer(

```

```

        T_obs > c_alpha
    )

    c(
        statistic = T_obs,
        critical_value = c_alpha,
        reject = reject
    )
}

## -----
## 10. Empirical power for one (n,r,d,theta) combination
## -----

estimate_power <- function(
    n,
    r,
    d,
    theta,
    R = 1000,
    alpha = 0.05,
    B = 500
) {

    rejection <- numeric(R)

    for (rep in 1:R) {

        result <-
            one_replication(
                n = n,
                r = r,
                d = d,
                theta = theta,
                alpha = alpha,
                B = B
            )

        rejection[rep] <-
            result["reject"]
    }
}

```

```

empirical_power <-
  mean(rejection)

## Monte Carlo standard error
MCSE <-
  sqrt(
    empirical_power *
      (1 - empirical_power) /
      R
  )

## Monte Carlo 95% interval
lower <-
  empirical_power -
  qnorm(0.975) * MCSE

upper <-
  empirical_power +
  qnorm(0.975) * MCSE

data.frame(
  n = n,
  r = r,
  d = d,
  theta = theta,
  alpha = alpha,
  R = R,
  B = B,
  empirical_power = empirical_power,
  MCSE = MCSE,
  lower_95 = max(0, lower),
  upper_95 = min(1, upper)
)
}

## -----
## 11. Run complete empirical-power experiment
## -----

power_results <- list()

```

```

counter <- 1

for (theta in theta_values) {

  for (n in n_values) {

    for (r in r_values) {

      for (d in d_values) {

        cat(
          "Running:",
          "theta =", theta,
          "n =", n,
          "r =", r,
          "d =", d,
          "\n"
        )

        power_results[[counter]] <-
          estimate_power(
            n = n,
            r = r,
            d = d,
            theta = theta,
            R = R,
            alpha = alpha,
            B = B
          )

        counter <- counter + 1
      }
    }
  }
}

```

```

## -----
## 12. Combine all results
## -----

```

```

power_results <-
  do.call(
    rbind,
    power_results
  )

rownames(power_results) <- NULL

## -----
## 13. Display empirical power
## -----

print(power_results)

## -----
## 14. Save results
## -----

write.csv(
  power_results,
  file = "empirical_power_r_d.csv",
  row.names = FALSE
)

## -----
## 15. Power tables for each theta
## -----

for (theta in theta_values) {

  cat(
    "\n===== \n"
  )

  cat(
    "Local alternative theta =",
    theta,
    "\n"
  )
}

```

```

cat(
  "=====\n"
)

temp <-
  subset(
    power_results,
    theta == !!theta
  )

print(temp)
}

## -----
## 16. Power matrix for a selected theta and n
## -----

theta_selected <- theta_values[1]

n_selected <- n_values[1]

power_matrix <-
  reshape(
    subset(
      power_results,
      theta == theta_selected &
      n == n_selected
    ),
    idvar = "r",
    timevar = "d",
    direction = "wide"
  )

print(power_matrix)

```